WordPress Performance Optimization: Unified Developer Reference

Version 1.2 — Released

Dan Knauss, General Editor

WordPress Performance Optimization: Unified Developer Reference

> Status: Released Version: 1.2 Date: 14 June 2026 General Editor: Dan Knauss
>
> Audience: This guide is for developers doing WordPress performance optimization who need to decide what to measure, where to look, what to change, and how to verify the result.
>
> Environments: Ordinary hosting, standard managed WordPress hosting platforms, and enterprise environments.
>
> Acknowledgements: This document has been checked against WordPress core behavior and official WordPress.org developer/administration documentation, including the WordPress.org Advanced Administration Handbook. It is also informed by and compared with Automattic/WordPress VIP Learn's Enterprise WordPress Performance course and Remkus de Vries' Make WordPress Fast course for the Within WordPress Guild. Community discussions around WordPress.org developer documentation on transients, object caching, cache bootstrap behavior, the Options API, and modern Core performance features have informed this document as well.
>
> Currency: Last verified against WordPress 7.0 on 2026-06-14.

This guide is written for the post-WordPress 7.0 baseline. It treats modern option autoload APIs, the WordPress 6.6 autoload vocabulary, Core speculative loading, WordPress 6.9 frontend/cron/cache changes, and WordPress 7.0 PHP/admin/editor/AI-connectors implications as part of the normal operating context rather than as add-ons. Version-specific guidance below is scoped to WordPress 7.0 unless a section explicitly names an older branch.

Table of Contents

1. Mental model: performance is the result of work

WordPress performance is not one number and not one layer. It is the cumulative result of work performed by:

1. DNS and connection setup.
2. TLS and HTTP negotiation.
3. CDN/edge cache.
4. Origin web server.
5. PHP runtime and OPcache.
6. WordPress bootstrap.
7. MU plugins, normal plugins, and theme code.
8. Database queries.
9. Object cache and transient storage.
10. Template rendering.
11. HTTP response headers.

12. Browser parsing, layout, paint, JavaScript, CSS, images, fonts, and third-party resources.
13. Follow-up requests such as REST, AJAX, media, tracking, personalization, and background calls.

The practical rule is simple:

> Performance improves when you remove work, cache work safely, move work out of the critical path, or make unavoidable work cheaper.

A fast site is not merely a site with a good Lighthouse score or a low TTFB. Users experience:

- how quickly useful content appears;
- how soon the page responds to interaction;
- whether layout remains stable;
- whether the page feels trustworthy and ready.

Backend metrics explain where delays originate. Frontend metrics explain how delays are experienced. You need both.

---

2. Source strengths and how to use them

`wordpress-performance-optimization-checklist.md` is strongest for

- whole-stack performance thinking;
- user-centric performance and Core Web Vitals;
- request-to-render diagnosis;
- DNS, TLS, and HTTP/2 vs HTTP/3 protocol tradeoffs;
- resource hints (`dns-prefetch`, `preconnect`, `preload`);
- frontend rendering, images, fonts, CSS, JavaScript, and third-party assets;
- plugin stack simplification;
- page builders and DOM bloat;
- WooCommerce and Action Scheduler considerations;
- Multisite resource amplification;
- DevTools and TRACE-style debugging;
- `wp-config.php` as policy (revisions, Heartbeat, debug constants);
- practical operator checklists and client communication.

`enterprise-performance-operational-checklist.md` is strongest for

- enterprise request-cycle analysis;
- cacheability, edge cache, TTL, invalidation, and cache misses;
- application-level bottlenecks;
- hooks, templates, uncached functions, AJAX, redirects, search, and sitemaps;
- `WP_Query` best practices at production scale;
- unbounded queries;
- taxonomy, post meta, `NOT IN`, `LIKE`, and SQL `EXPLAIN`;
- Elasticsearch/OpenSearch-style query offloading;
- partial-output / fragment caching with `ob_start()` + object cache;
- object-cache race conditions and cache stampedes;
- traffic patterns, high-traffic event prep, load testing, New Relic, and alerting.

Query Monitor appears throughout the source material and belongs to both guide contexts: use it for general diagnosis and for enterprise-scale backend/cache/database investigations.

WordPress.org developer docs are authoritative for

- Transients API semantics;
- `get_transient()` and `set_transient()` behavior;
- `WP_Object_Cache` behavior;
- `wp_start_object_cache()` and object-cache drop-in loading;
- distinction between `advanced-cache.php`, `object-cache.php`, and `WP_CACHE`;
- the WP 6.6+ Options API autoload vocabulary (`on`/`off`/`auto`/`auto-on`/`auto-off`);
- the Speculation Rules / speculative loading work merged into Core 6.8;
- WP-Cron alternatives and the system task scheduler integration.

Advanced Admin Handbook PR clarification is strongest for

- explaining that `WP_CACHE` is not a general “turn caching on” switch;
- distinguishing `advanced-cache.php` page-cache drop-ins from `object-cache.php` persistent object-cache drop-ins;
- clarifying that transients are not always memory-backed.

Naming boundary for source material

The two main repository-owned documents are the general `wordpress-performance-optimization-checklist.md` and the enterprise `enterprise-performance-operational-checklist.md`. Source names such as Remkus, Make WordPress Fast,

WordPress VIP, and Enterprise WordPress Performance are used only for acknowledgements, source notes, and correction rationale — not as titles for the guides themselves.

---

3. Optimization workflow

Use this sequence for almost every performance investigation.

Step 1: Define the symptom

Classify the problem before touching code.

- High TTFB?
- Slow cached page?
- Slow uncached page?
- Slow admin screen?
- Slow REST endpoint?
- Slow `admin-ajax.php` call?
- Poor LCP?
- Poor INP?
- Poor CLS?
- Search is slow?
- Sitemap generation is slow?
- Cron/background jobs cause spikes?
- Site fails during traffic surges?

Step 2: Identify the request context

Record:

- exact URL or route;
- logged-in or anonymous;
- cacheable or intentionally dynamic;
- device/network conditions;
- expected traffic level;
- whether the issue is constant, sporadic, traffic-dependent, or deployment-dependent.

Step 3: Capture a baseline

Do not optimize from memory.

```
curl -s -o /dev/null -D headers.txt \
  -w "dns:%{time_namelookup} connect:%{time_connect} tls:%{time_appconnect} ttfb:%
   https://example.com/target/
```

Record:

Target	User state	Expected cacheability	Cache status	TTFB	LCP	INP	CLS	Notes
`/`	anonymous	cacheable						
`/shop/`	anonymous	cacheable						
`/wp-json/...`	varies	endpoint-specific						

Step 4: Decide whether WordPress should run

For anonymous public pages, ask first:

- Can this HTML be served from edge/page cache?
- Did it actually hit cache?
- If it missed or bypassed cache, why?

If the page should be cacheable but WordPress is executing, fix cacheability before optimizing PHP or SQL.

Step 5: If WordPress runs, locate the dominant cost

Check:

- WordPress bootstrap and hooks;
- plugin/theme callbacks;
- database query count and slow queries;
- object-cache hit rate;
- transient usage;
- external HTTP requests;
- template rendering;
- REST/AJAX follow-up requests;
- cron/background tasks.

Step 6: If the server is fast but the page feels slow, inspect the browser

Check:

- LCP element and timing;
- render-blocking CSS;
- blocking or long-running JavaScript;
- image size and priority;
- font loading;
- layout shifts;
- third-party scripts;
- DOM size and page-builder output;
- main-thread work in DevTools.

Step 7: Change one thing and verify

Every change needs before/after evidence. If the measured bottleneck does not move, you either fixed the wrong thing or measured the wrong target.

---

4. Cache layers and responsibilities

Caching is not one mechanism. Each layer solves a different problem.

Layer	Typical mechanism	What it avoids	Common failure
Browser cache	HTTP headers	re-downloading static assets	bad headers or unversioned assets
CDN/edge cache	CDN, edge rules	long-distance origin trips	cookies/query strings bypass cache
Full-page cache	edge/server/plugin page cache	PHP + database execution	stale content, broad purges, personalization
Object cache	Redis/Memcached via `object-cache.php`	repeated DB/application work	poor hit rate, evictions, stampedes

Layer	Typical mechanism	What it avoids	Common failure
Transients	Transients API	repeated expensive computation	unaware of DB fallback, too many keys, stale data
OPcache	PHP OPcache	recompiling PHP	stale bytecode, memory exhaustion
Runtime/request cache	static variables, non-persistent object cache	repeated work in one request	mistaken for cross-request persistence

The transients “failure” is not the database fallback itself — the fallback is by design. The failure is writing transient-heavy code without checking whether a persistent object cache is present, so the database silently absorbs work it should not have to do.

Important distinction

```
define( 'WP_CACHE', true );
```

`WP_CACHE` allows WordPress to load:

`wp-content/advanced-cache.php`

This is commonly used by page-cache drop-ins.

Persistent object caching is different and uses:

`wp-content/object-cache.php`

`WP_CACHE` does not enable Redis, Memcached, browser caching, or OPcache by itself.

---

5. Full-page cache and edge cache

Full-page caching is often the largest backend performance win because it can prevent WordPress from running at all.

Cacheable candidates

- homepage;
- posts and pages;
- category/tag archives;
- landing pages;
- public product/archive pages when not personalized;
- public documentation pages;
- anonymous GET endpoints that return identical data.

Bypass candidates

- WordPress admin;
- logged-in personalized views;
- cart;
- checkout;
- account pages;
- previews;
- nonce-sensitive forms;
- private API responses;
- pages that vary by permission, session, cart, or account state.

Cache miss triage

If a page should be cached but misses, check:

- `Set-Cookie` headers;
- `Cache-Control: no-store`, `private`, or overly conservative directives;
- `Vary: Cookie` or unnecessary variation;
- query strings such as `utm_*`, `fbclid`, `gclid`;
- authentication headers;
- short TTLs;
- frequent full-site purges;
- plugin rules that mark the response uncacheable;
- CDN/origin rule mismatch.

Query-string fragmentation examples

```
/shop?utm_source=newsletter&utm_medium=email&utm_campaign=spring_sale
/shop?fbclid=IwAR1XyZExampleValue
/shop?gclid=EAIaIQobChMIExample
```

```
/shop?utm_source=facebook&utm_content=carousel_ad_3
/shop?lang=en
/shop?preview=true
/shop?utm_source=newsletter&utm_medium=email&fbclid=IwAR1XyZExampleValue
```

Normalize or ignore marketing parameters when they do not change content. Do not normalize parameters that actually change language, currency, region, personalization, previews, or permissions.

Platform warning

On enterprise platforms with edge caching, do not blindly add plugin page caching. Combining plugin page cache with platform edge cache can complicate invalidation and increase stale-content risk. Prefer one authoritative full-page cache layer per responsibility.

---

6. HTTP headers, TTL, and invalidation

Headers define cache behavior.

Inspect:

```
curl -I https://example.com/page/
```

Review:

- `Cache-Control`;
- `Expires`;
- `Vary`;
- `Set-Cookie`;
- CDN cache status headers;
- redirect headers;
- security headers that might affect resource loading.

TTL guidance

- Use long TTLs for versioned static assets.
- Use shorter TTLs for frequently changing HTML.
- Prefer precise invalidation over universally short TTLs.
- Avoid full-site purges for small content changes when targeted purges are available.
- Watch for invalidation storms during deploys or high-traffic events.

Redirect diagnostics

```
curl -IL https://example.com/some-url/
```

Look for `x-redirect-by` (the diagnostic pattern below originates in WordPress VIP's operational guidance; see enterprise operational checklist §4):

```
x-redirect-by: WordPress
x-redirect-by: Polylang Pro
```

Debugging redirect source in a safe environment:

```
add_filter( 'x_redirect_by', function( $x_redirect_by, $status, $location ) {
    wp_die( var_dump( array(
        $x_redirect_by,
        $status,
        $location,
        wp_debug_backtrace_summary(),
    ) ) );
}, 10, 3 );
```

---

7. DNS, TLS, and HTTP transport

A page cannot be fast if connection setup is slow. The transport layer is the first place a request spends time, and a misconfigured edge can add hundreds of milliseconds before WordPress even runs.

DNS

- Use a reputable DNS provider with fast global resolution (Anycast routing is the standard).
- Avoid long CNAME chains; each indirection adds a lookup.
- Remove stale records.
- Lower TTLs before planned migrations; raise them for stable records.
- Confirm IPv4 and IPv6 behavior are both intentional.

TLS

- Enforce HTTPS site-wide and end-to-end (origin and CDN).
- Use TLS 1.3 where supported — it cuts a round-trip compared to TLS 1.2.
- Enable session resumption and OCSP stapling at the edge to reduce handshake cost on repeat visits.
- Enable HSTS only after HTTPS is fully correct across subdomains.
- Keep certificates auto-renewing; expired certs are a uniquely catastrophic outage.

HTTP/2 vs HTTP/3

- HTTP/2 multiplexes streams over one TCP connection and removes head-of-line blocking at the application layer; it is a baseline expectation for production WordPress in 2026.
- HTTP/3 runs over QUIC (UDP) and removes head-of-line blocking at the transport layer too. It helps most on lossy or mobile networks.
- Most CDNs support both; verify that your edge actually negotiates HTTP/3 (`alt-svc` advertised, then a follow-up request actually using `h3`).

Redirect chains

A single redirect costs a round-trip. Two or three chained redirects can dominate TTFB for the first paint.

- Remove `http://` -> `https://` chains by configuring HTTPS directly at the edge.
- Resolve apex vs `www` canonicalization in one hop.
- Resolve trailing-slash canonicalization in one hop.
- Verify with `curl -IL`.

Useful checks

```
curl -I -L https://example.com/
curl -w "namelookup:%{time_namelookup} connect:%{time_connect} tls:%{time_appconne
  -o /dev/null -s https://example.com/
```

---

8. Resource hints: `dns-prefetch`, `preconnect`, `preload`

Resource hints tell the browser to start work before it would otherwise. Used precisely, they shave hundreds of milliseconds off LCP. Used carelessly, they create connection

storms and waste mobile bandwidth.

`dns-prefetch`

Resolves the DNS for an origin in advance. Cheap; safe to use for any third-party origin the page will actually contact.

```
<link rel="dns-prefetch" href="//fonts.gstatic.com">
```

`preconnect`

Resolves DNS and completes the TCP and TLS handshake in advance. More expensive than `dns-prefetch`, but pays off for origins the browser will hit early.

```
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
```

Use `crossorigin` for origins that serve credentialed or CORS resources (fonts are the classic example). Without it, the preconnect won't be reused for the font fetch and you pay the cost twice.

`preload`

Tells the browser to download a specific resource at high priority. Reserve this for resources that are (a) on the critical rendering path and (b) discovered late by the parser.

```
<link rel="preload" href="/fonts/inter-var.woff2" as="font" type="font/woff2" cros
```

For the LCP image specifically, use `imagesrcset` and `imagesizes` so the preload respects the responsive image strategy:

```
<link rel="preload" as="image"
      href="/hero-1600.webp"
      imagesrcset="/hero-800.webp 800w, /hero-1600.webp 1600w"
      imagesizes="100vw">
```

Overuse warnings

- Don't preconnect to more than ~3–4 origins per page. Each open connection has CPU and memory cost on the client.
- Don't preload anything the browser would have discovered on its own from the HTML — that just shifts the cost earlier and clutters the network tab.
- Preloaded fonts must match the `type` and `crossorigin` of the actual fetch, or the browser will fetch the font twice.

Modern alternative: speculative loading

For navigation hints (not resource hints), WordPress 6.8+ ships Speculation Rules support whose Core default is conservative prefetching, while prerender is an opt-in/plugin/custom configuration choice. Treat speculative loading as part of cache and analytics design: exclude authenticated, session-keyed, cart/checkout, preview, and other personalized paths before widening prefetch or prerender behavior.

---

9. PHP runtime, OPcache, and worker capacity

When WordPress executes, PHP runtime configuration sets the floor on how fast it can possibly run.

OPcache

OPcache caches compiled PHP bytecode in memory so each request does not re-parse and re-compile every PHP file. It is the single largest PHP-level perf win, and it is often misconfigured.

- Confirm OPcache is enabled for web requests (check `phpinfo()` or a Site Health module).
- Confirm `opcache.memory_consumption` is not exhausted. A full cache silently degrades — old entries get evicted and re-compiled on each request.
- Confirm `opcache.max_accelerated_files` is large enough for your codebase (WordPress core alone has ~3,000 PHP files; a heavy WooCommerce site can pass 30,000).
- Set `opcache.validate_timestamps` based on your deploy workflow: leave it on (with a sane `revalidate_freq`) for hosts where files change in place; turn it off only if you reset OPcache as part of deploys.

```
opcache.enable=1
opcache.memory_consumption=256
opcache.max_accelerated_files=20000
opcache.validate_timestamps=1
opcache.revalidate_freq=60
; opcache.jit_buffer_size=100M ; leave off unless benchmarks prove a WordPress benef
```

Note on `opcache.jit_buffer_size`: PHP's JIT compiler is not a clear win for WordPress web-request workloads in most measurements. It consumes memory that may be more valuable for OPcache bytecode storage or PHP-FPM workers. Leave JIT off unless representative benchmarks of the specific site — cached and uncached paths, p95 TTFB, worker saturation, and OPcache memory headroom — show a measurable benefit.

PHP workers

PHP workers (php-fpm pool size, Apache MPM workers, etc.) set the concurrency ceiling. If you run out of workers, requests queue and TTFB collapses for everyone — even cached pages can stall if the CDN is forced back to origin.

- Size workers based on observed concurrent uncached requests, not total traffic. A site that fully edge-caches 99 % of traffic needs few workers; one that runs PHP on every request needs many.
- Monitor worker saturation as a primary signal, not just CPU.
- If workers are spending time on slow plugins or external HTTP calls, the fix is to shorten those calls (or move them to background work) rather than adding workers indefinitely.

PHP version

Use a supported PHP version. WordPress 7.0 raises the minimum supported PHP version to 7.4.0, while PHP 8.3 remains the recommended baseline for modern performance work; PHP 8.3/8.4 generally provide meaningful performance and security wins over 7.x without WordPress code changes. Older PHP is slower, less secure, and increasingly unsupported by plugin authors.

---

10. WordPress runtime and plugin/theme cost

When WordPress executes, every plugin, theme, hook, query, and external call can add cost.

Developer checklist

- Are hooks scoped to only the contexts that need them?
- Does code run on every request when it only belongs on one template?
- Are admin-only features loading on the frontend?
- Are frontend assets loaded globally?
- Are queries inside loops?
- Are remote HTTP calls made during page rendering?
- Are expensive computations cached?
- Are plugin modules disabled when unused?
- Are multiple optimization plugins modifying the same assets or cache layers?

Hook profiling

Use Query Monitor, WP-CLI profile, APM traces, or targeted timing to identify expensive callbacks.

Availability check. `wp profile` and `wp doctor` are not bundled with every WP-CLI install; they ship as separate command packages. Check availability before using them:

```
wp cli has-command profile && echo "profile available"
wp cli has-command doctor && echo "doctor available"
# Install if missing and your environment allows package installs:
# wp package install wp-cli/profile-command
# wp package install wp-cli/doctor-command
```

If neither command nor package install is available (common on managed hosts), fall back to Query Monitor, APM traces, host-provided profilers, or in-code `microtime()` timers.

```
wp profile stage --url="https://example.com/target/"
wp profile hook --url="https://example.com/target/" --all
wp doctor check
```

Plugin-stack guidance

Plugin count is not the performance metric. Plugin behavior is.

Review:

- global asset loading;
- database queries;

- remote calls;
- cron jobs;
- autoloaded options;
- cache bypass headers;
- duplicate features across plugins;
- page-builder extensions and widget libraries;
- security and maintenance quality.

---

11. Database performance

Database optimization is not “cleaning the database.” It is reducing expensive reads/writes and making unavoidable queries efficient.

Primary signals

- high query count;
- slow queries;
- duplicate queries;
- full table scans;
- filesorts;
- temporary tables;
- unbounded result sets;
- high autoloaded option size;
- repeated queries that should be cached;
- search/filtering that belongs in a search index.

`WP_Query` best practices

Use explicit limits:

```
$query = new WP_Query(
    array(
        'post_type'      => 'post',
        'posts_per_page' => 10,
        'orderby'        => 'date',
        'order'          => 'DESC',
        'no_found_rows'  => true,
    )
);
```

The `no_found_rows => true` flag is mainstream WordPress core guidance, not a WordPress VIP-specific or enterprise-only recommendation. It suppresses the `SQL_CALC_FOUND_ROWS` clause that powers pagination counts; if you are not using pagination, skipping it is a meaningful win on large `wp_posts` tables. (See WordPress Trac #10469 for the history of why `SQL_CALC_FOUND_ROWS` is now considered an anti-pattern.)

Avoid unbounded queries:

```
'posts_per_page' => -1
```

This may appear harmless in development and fail at production scale.

Dangerous patterns

- broad meta queries;
- sorting by unindexed meta values;
- high-cardinality taxonomy filtering;
- large `NOT IN` lists;
- leading-wildcard `LIKE`;
- search implemented with SQL over large content tables;
- accidental broad queries from falsey IDs;
- `switch_to_blog()` in loops on Multisite;
- repeated queries in template partials.

`NOT IN` example to scrutinize

```
SELECT * FROM wp_posts
WHERE post_type = 'post'
  AND post_status = 'publish'
  AND ID NOT IN (1, 2, 3);
```

Large exclusions often scale poorly. Prefer positive selection, precomputed sets, or different data modeling when possible.

`LIKE` examples

```
SELECT * FROM wp_posts WHERE post_title LIKE '%WordPress%';
SELECT * FROM wp_posts WHERE post_title LIKE 'WordPress%';
```

The second form is more index-friendly than a leading wildcard, but serious search workloads should usually move to dedicated search infrastructure.

`EXPLAIN`

Use `EXPLAIN` on slow SQL:

```
EXPLAIN SELECT * FROM wp_posts WHERE post_date > '2023-01-01';
EXPLAIN
SELECT wp_posts.*, wp_postmeta.meta_value
FROM wp_posts
JOIN wp_postmeta ON wp_posts.ID = wp_postmeta.post_id
WHERE wp_posts.post_date > '2023-01-01';
```

Inspect:

- table;
- access type;
- possible keys;
- key actually used;
- estimated rows;
- `Extra` details such as filesort or temporary table usage.

---

12. Autoloaded options and `wp_options` (updated for WP 6.6+)

Autoloaded options are loaded early and can affect every request.

What changed in WordPress 6.6

Before WordPress 6.6 (June 2024), the `wp_options.autoload` column held one of two string values: `'yes'` or `'no'`. As of WordPress 6.6, newly written options use a richer vocabulary:

Value	Meaning
`'on'`	Explicitly marked to autoload (caller passed `true`).
`'off'`	Explicitly marked not to autoload (caller passed `false`).

Value	Meaning
`'auto'`	Caller did not pass a value; WordPress decides. Currently autoloads in 6.6; default may change.
`'auto-on'`	Dynamically determined to autoload.
`'auto-off'`	Dynamically determined not to autoload (e.g., the option is too large).

Older `'yes'`/`'no'` rows still exist on upgraded sites and are treated as `'on'`/`'off'`. There is no migration; both vocabularies coexist indefinitely.

WP 6.6 also added automatic skip-autoload behavior for large options and a Site Health check ("Autoloaded options could affect performance") that warns when total autoload size exceeds ~800 KB.

Review queries (6.6-aware)

Old guidance filters a `WHERE autoload` filter that matches only `'yes'`. That misses everything written under the new vocabulary. Use this instead:

```
SELECT option_name, LENGTH(option_value) AS size, autoload
FROM wp_options
WHERE autoload IN ('yes', 'on', 'auto', 'auto-on')
ORDER BY size DESC
LIMIT 20;
```

Total autoload footprint:

```
SELECT SUM(LENGTH(option_value)) AS autoload_bytes
FROM wp_options
WHERE autoload IN ('yes', 'on', 'auto', 'auto-on');
```

Changing autoload state

Don't hand-edit the `autoload` column. Use the WP 6.4+ API:

```
wp_set_option_autoload( 'my_huge_option', false );      // singular
wp_set_options_autoload( array( 'opt_a', 'opt_b' ), false ); // bulk
```

Or the WP-CLI command:

```
wp option set-autoload my_huge_option no
```

Guidance

- Autoload only small, frequently needed options.
- Do not autoload large serialized blobs.
- Do not treat `wp_options` as a general cache table.
- Watch the Site Health check; address it before it becomes critical.
- Remove orphaned plugin options only after backup and verification.
- Changing autoload flags can break plugins if done blindly; test on staging.

---

13. Transients and object cache

Transients are temporary cached values with expiration, but their backend depends on the environment.

Core rule

- With persistent object cache: transients can be stored in Redis/Memcached/object cache.
- Without persistent object cache: transients fall back to the database, typically `wp_options`.

Therefore:

> Transients are a cache abstraction with a database fallback, not guaranteed memory storage.

This is the design, not a bug. The bug is writing transient-heavy code without checking which backend is in use.

Database-backed transient row pairs

When transients fall back to the database, an expiring transient typically writes two option rows:

```
_transient_{name}
_transient_timeout_{name}
```

The `_timeout_` row stores the Unix epoch at which the transient expires. On Multisite, network-wide transients use analogous `_site_transient_{name}` / `_site_transient_timeout_{name}` keys (see §24).

The falsey-value pitfall

`get_transient()` returns `false` for both a cache miss and an expired transient. If the cached value itself can legitimately be `false`, `0`, an empty string, or `null`, you cannot distinguish a hit from a miss using the return value alone. Wrap the value or use a sentinel:

```
$wrapped = get_transient( 'myplugin_result' );

if ( false === $wrapped ) {
   $value   = myplugin_compute();        // could legitimately be 0 or ''
    $wrapped = array( 'value' => $value );
    set_transient( 'myplugin_result', $wrapped, HOUR_IN_SECONDS );
}

$value = $wrapped['value'];
```

Safe transient pattern

```
$cache_key = 'myplugin_expensive_result_' . md5( $context_id );
$result    = get_transient( $cache_key );

if ( false === $result ) {
    $result = myplugin_build_expensive_result( $context_id );
    set_transient( $cache_key, $result, HOUR_IN_SECONDS );
}
```

Use transients for

- expensive query results;
- remote API responses;
- rendered fragments;
- computed values that can be regenerated;
- short-lived shared results.

Avoid transients for

- permanent settings;
- canonical data;
- values that cannot disappear early;
- huge blobs on sites without persistent object cache;
- per-request or high-cardinality keys;
- private data under shared keys;
- data that changes so frequently it rarely hits cache.

Inspect database-backed transients

```
SELECT option_name, LENGTH(option_value) AS size, autoload
FROM wp_options
WHERE option_name LIKE '\_transient\_%'
ORDER BY size DESC
LIMIT 20;
SELECT COUNT(*) AS transient_rows,
       SUM(LENGTH(option_value)) AS transient_bytes
FROM wp_options
WHERE option_name LIKE '\_transient\_%';
```

Object cache monitoring

Persistent object cache is valuable only if hit rates are healthy and memory pressure is controlled. Monitor:

- hit rate;
- miss rate;
- evictions;
- memory usage;
- slow cache operations;
- cache flush frequency;

- hot keys;
- network/service errors.

Redis vs Memcached, and Object Cache Pro

WordPress works with both Redis and Memcached as persistent object cache backends, but the choice has practical consequences:

- Memcached is a pure in-memory key/value store. Fast, simple, ephemeral. Eviction is LRU; there is no persistence and no advanced data structures. This is the historical default on WordPress VIP and similar enterprise platforms.
- Redis offers richer data structures, optional persistence, pub/sub, atomic operations, and better introspection.

Backend selection. Use the host-supported `object-cache.php` drop-in first. Absent a platform constraint, Redis is generally a better match for WordPress workloads because of richer observability (per-key TTLs, hot-key detection), data structures useful for cache groups, and atomic operations used by some lock patterns. Memcached is appropriate when it is the platform-supported, well-instrumented default — switching backends has operational cost and should be justified by measured cache hit rate, latency, eviction, and failure behavior on the target host.

For the integration layer, Object Cache Pro (commercial) is the most common production-grade Redis drop-in for WordPress, offering monitoring, async writes, group prefetching, and per-group serialization choices. Free alternatives include the Redis Object Cache plugin and host-provided drop-ins.

Whichever backend you choose, the drop-in lives at `wp-content/object-cache.php` and is loaded early in the WordPress boot.

---

14. Race conditions and cache stampedes

Cache stampedes happen when many requests miss the same expensive cache key and all regenerate it simultaneously.

Object-cache form

These lock patterns assume a persistent object cache with atomic add semantics, such as Redis or Memcached via `wp-content/object-cache.php`. On default WordPress with no persistent object cache, `wp_cache_add()` is request-local: every concurrent request

can “acquire” the lock simultaneously, so it provides no cross-request coordination. Confirm a persistent object cache is in place before relying on this pattern.

Fallbacks when persistent object caching is not available:

- Use a database-backed mutex or a short-lived transient/option lock, accepting the database-write cost and failure modes.
- Use a platform-provided distributed lock service if the host offers one.
- Use stale-while-revalidate: serve the previous value while a warmer regenerates asynchronously.
- Pre-warm hot keys outside user requests with cron, deploy hooks, or scheduled actions.
- Restructure so the expensive operation never runs inside a user request.

```
$cache_key = 'myplugin_expensive_data';
$lock_key  = 'myplugin_expensive_data_lock';

$data = wp_cache_get( $cache_key, 'myplugin' );

if ( false === $data ) {
    $lock_acquired = wp_cache_add( $lock_key, 1, 'locks', 30 );

    if ( $lock_acquired ) {
        $data = myplugin_generate_expensive_data();
       wp_cache_set( $cache_key, $data, 'myplugin', HOUR_IN_SECONDS );
        wp_cache_delete( $lock_key, 'locks' );
    } else {
        $data = myplugin_get_safe_fallback();
    }
}
```

Transient form

When you specifically want the transient layer, remember that storing the result in a transient does not make the `wp_cache_add()` lock cross-request safe on sites without a persistent object cache. If the lock precondition above is not met, use one of the fallback strategies instead of this pattern:

```
$cache_key = 'myplugin_expensive_data';
$lock_key  = 'myplugin_expensive_data_lock';

$data = get_transient( $cache_key );
```

```
if ( false === $data ) {
    $lock_acquired = wp_cache_add( $lock_key, 1, 'locks', 30 );

    if ( $lock_acquired ) {
        $data = myplugin_generate_expensive_data();
        set_transient( $cache_key, $data, HOUR_IN_SECONDS );
        wp_cache_delete( $lock_key, 'locks' );
    } else {
        $data = myplugin_get_safe_fallback();
    }
}
```

The lock's TTL is only a safety property after the lock is shared across requests. Without a persistent object cache, the lock is only request-local and does not prevent a stampede.

Additional tactics

- add TTL jitter so keys do not all expire at once;
- refresh hot keys before expiry (proactive regeneration);
- serve stale while regenerating where acceptable;
- batch object-cache requests with `wp_cache_get_multiple()`;
- avoid cache calls in tight loops when one multi-get would work.

---

15. REST API, AJAX, and follow-up requests

Cached HTML can still be followed by uncached dynamic requests.

Check for:

- `/wp-json/...` calls;
- `/wp-admin/admin-ajax.php` calls;
- WooCommerce cart fragments;
- personalization endpoints;
- search/autocomplete;
- analytics or marketing calls;
- polling.

REST guidance

- Public GET endpoints returning identical data can be cache candidates.
- Authenticated endpoints usually bypass edge cache.
- Nonces and cookies introduce state.
- Expensive endpoint callbacks should cache internally.
- Avoid polling when event-driven updates are possible.

AJAX guidance

- Avoid `POST` for cacheable reads.
- Prefer REST endpoints over `admin-ajax.php` for structured APIs.
- Cache public GET responses when safe.
- Move expensive work out of synchronous user requests.

---

16. WP-Cron, Action Scheduler, and background work

Request-triggered WP-Cron can create unpredictable user-facing latency. On production and high-traffic sites, move due-event execution to a real scheduler — but the order matters.

Production pattern

Sequence matters. Set up and verify the external runner *before* disabling request-triggered cron. If you flip the order, scheduled events stop running until the runner is in place.

Install an external runner. Pick the interval based on your shortest acceptable lag — one minute for jobs that need to be timely, five minutes for general housekeeping:

```
*/5 * * * * cd /path/to/site && wp cron event run --due-now --quiet
```

Verify the runner is firing before changing `wp-config.php`:

```
wp cron event list --due-now
# Check cron, PHP, and application logs for the runner invocation.
```

Only after verification, disable request-triggered cron:

```
define( 'DISABLE_WP_CRON', true );
```

If WP-CLI is not available, hit the cron endpoint over HTTP. Either `curl` or `wget` works:

```
*/5 * * * * curl -s https://example.com/wp-cron.php?doing_wp_cron >/dev/null 2>&1
*/5 * * * * wget -q --delete-after https://example.com/wp-cron.php?doing_wp_cron
```

After setting the constant, confirm scheduled events continue to complete on time. For WooCommerce and Action Scheduler-heavy sites, monitor the Action Scheduler queue because backlog can build silently if the external runner stalls.

Rollback: comment out the `DISABLE_WP_CRON` line, clear OPcache if needed, and confirm due events begin progressing again.

Interval: Enterprise contexts often use `* * * * *` / every minute instead of `*/5 * * * *` / every five minutes. The right answer depends on what is scheduled. If you have second-counts-late jobs, run every minute; otherwise five minutes is the calmer default.

Review

17. `wp-config.php` as policy

Defaults are safe, not optimal. `wp-config.php` is where you encode site-level performance and operational policy. Treat wp-config as policy, not as a place to dump constants you copied from somewhere else.

```
// Debug — production-safe.
define( 'WP_DEBUG', false );
define( 'WP_DEBUG_LOG', false );
define( 'SCRIPT_DEBUG', false );

// Editorial — limit revision sprawl and trash retention.
define( 'WP_POST_REVISIONS', 10 );          // or false to disable
define( 'AUTOSAVE_INTERVAL', 120 );          // seconds
define( 'EMPTY_TRASH_DAYS', 14 );

// Cron — only after an external runner has been installed and verified (see §16).
define( 'DISABLE_WP_CRON', true );

// File modifications — disable on platforms that deploy code via CI/CD.
```

```
define( 'DISALLOW_FILE_MODS', true );

// Cache — page-cache drop-in opt-in (see §4 for what this does NOT enable).
define( 'WP_CACHE', true );
```

Heartbeat API

The Heartbeat API powers admin features like autosave and dashboard widgets but polls aggressively. On large editorial sites it can be a real source of admin-side load.

```
// Slow Heartbeat on the frontend; keep it normal in the editor.
add_filter( 'heartbeat_settings', function( $settings ) {
    if ( ! is_admin() ) {
        $settings['interval'] = 60;
    }
    return $settings;
} );
```

XML-RPC

Unused on most modern sites. Disable if you do not use Jetpack, the WordPress mobile app, or remote-publishing clients.

```
add_filter( 'xmlrpc_enabled', '__return_false' );
```

Emojis and embeds

Both inject scripts and styles on the frontend. Removable on sites that don't need them; the savings are small but cumulative.

```
remove_action( 'wp_head', 'print_emoji_detection_script', 7 );
remove_action( 'wp_print_styles', 'print_emoji_styles' );
remove_action( 'wp_head', 'wp_oembed_add_discovery_links' );
```

Use these with judgment. Don't paste configuration blindly — align it with editorial, hosting, and maintenance needs.

---

18. Partial-output / fragment caching

When full-page caching is impossible (logged-in views, mixed content, frequently changing components) but a specific expensive fragment is reused across many requests, cache the fragment. Our enterprise operational checklist §7 calls this out explicitly; the pattern is `ob_start()` plus the object cache.

```
<?php
$role_fragment = is_user_logged_in() ? 'logged_in' : 'logged_out';
$locale        = determine_locale();
$cache_key     = sprintf(
    'recent_posts_sidebar_v2_blog%d_locale%s_state%s',
    get_current_blog_id(),
    sanitize_key( $locale ),
    $role_fragment
);

$recent_posts_html = wp_cache_get( $cache_key, 'page_fragments' );

if ( false === $recent_posts_html ) {
    ob_start();
    // Generate the expensive sidebar output here.
    get_template_part( 'partials/recent-posts-sidebar' );
    $recent_posts_html = ob_get_clean();

    wp_cache_set(
        $cache_key,
        $recent_posts_html,
        'page_fragments',
        HOUR_IN_SECONDS
    );
}

echo $recent_posts_html;
```

Invalidate this cache group or version the key on relevant source changes, such as `save_post` for content-backed fragments.

Cache-key safety checklist for fragment / partial-output caching

Before caching a fragment with `wp_cache_set()` or `set_transient()`, verify that the key includes every dimension that varies the output:

- Site / blog scope: `get_current_blog_id()` on multisite.

- Locale: `get_locale()` or `determine_locale()` if locale affects content.
- Currency / region: any active currency, region, or store context.
- Auth state: logged-in vs logged-out as separate keys.
- User identity: only when the fragment is intentionally per-user and you have evaluated privacy and cardinality implications.
- Role / capability: if the fragment differs by role or capability, include the relevant subset.
- A/B / personalization bucket: include the bucket identifier.
- Query parameters that change output: explicit allowlist, never raw `$_GET`.

Privacy and cardinality:

- Never cache personalized output under a shared key. One shared-key mistake can leak one user's content to another.
- Per-user keys grow cache footprint linearly with active users. Bound this with short TTLs and use them only when regenerate cost dominates.
- On persistent object caches, monitor key cardinality and memory pressure for any per-user fragment cache group.

Invalidation triggers:

- Source content change: `save_post`, custom post type hooks, ACF/CMB2 update hooks.
- User profile / role / capability change: `set_user_role`, `profile_update`.
- Locale / settings change.
- Multisite blog switch / domain change.

Guidance:

- Pick cache keys that include only the variables that actually change output.
- Set an expiration that matches freshness requirements; invalidate on relevant `save_post` / data-change hooks if necessary.
- Never cache personalized fragments under a shared key. If the fragment must vary per user, use a per-user key only after reviewing privacy, cardinality, TTL, and invalidation behavior.
- Use cache groups (`'page_fragments'` above) to make targeted flushes possible.

19. Frontend and browser performance

A fast origin response does not guarantee a fast experience.

Core Web Vitals thresholds

Metric	Good	Needs improvement	Poor
LCP — Largest Contentful Paint	<= 2.5 s	2.5–4.0 s	> 4.0 s
INP — Interaction to Next Paint	<= 200 ms	200–500 ms	> 500 ms
CLS — Cumulative Layout Shift	<= 0.1	0.1–0.25	> 0.25

Thresholds are evaluated at the 75th percentile across a site's real users. INP replaced FID as a Core Web Vital on March 12, 2024.

LCP checklist

- Identify the LCP element.
- Improve TTFB if HTML arrives late.
- Optimize the LCP image.
- Do not lazy-load the LCP image.
- Set dimensions.
- Reduce render-blocking CSS/JS.
- Preload only truly critical resources (see §8).

INP checklist

- Reduce long JavaScript tasks (anything over 50 ms is suspect).
- Remove unnecessary third-party scripts.
- Defer or delay non-critical JavaScript.
- Avoid heavy page-builder runtime logic.
- Reduce DOM size and layout recalculation.
- Split work and avoid blocking the main thread.

CLS checklist

- Set image/video dimensions.
- Reserve space for ads, embeds, banners, notices.
- Stabilize font loading.

- Avoid injecting content above existing content after render.

Images

- Use correct sizes.
- Use `srcset` and `sizes`.
- Compress appropriately.
- Use WebP/AVIF where suitable; the Performance Lab ecosystem currently exposes this through Modern Image Formats (formerly named for WebP uploads). Verify the active plugin and image pipeline before rollout.
- Lazy-load below-fold images. (Core auto-adds `loading="lazy"` to most `<img>` tags.)
- Preload the true LCP image only when necessary.

Fonts — FOIT vs FOUT

When a browser needs a font it doesn't yet have, it picks one of two unhappy states:

- FOIT — Flash Of Invisible Text: text is hidden until the font arrives. Bad LCP, but no shift.
- FOUT — Flash Of Unstyled Text: fallback font renders, then swaps when the web font arrives. Good LCP, but causes layout shift (a CLS hit).

Control the choice with `font-display`:

- `font-display: swap` — fast first paint, accepts CLS. Usually the right call for body text.
- `font-display: optional` — fast first paint, only uses the web font if it arrives quickly. Best for sites where layout stability matters more than typography fidelity.
- `font-display: block` — short FOIT, then swap. Use only when the brand font is visually critical and you can self-host with minimal latency.

Other font guidance:

- Reduce families, weights, and variants ruthlessly. Each is a separate download.
- Self-host when it improves control and caching (`fonts.gstatic.com` adds a third-party DNS/TLS round-trip — see §7).
- Preload only critical above-fold fonts, with matching `type` and `crossorigin` (see §8).

CSS and JavaScript

- Remove unused assets.

- Scope enqueues to pages that need them.
- Avoid duplicate libraries.
- Defer non-critical scripts (`defer`/`async`).
- Avoid broad minify/combine settings that break dependencies.
- Trace every asset back to the plugin/theme that enqueued it.

Critical CSS

Inline a small block of CSS sufficient to render above-the-fold content, then load the rest asynchronously. This shortens LCP at the cost of some maintenance complexity.

- Generate critical CSS from the actual rendered viewport, not from a manual guess.
- Keep critical CSS small. The ~14 KB figure is a rule of thumb derived from the standard initial congestion window (10 x MSS approximately 14 KB across multiple segments), not a single-packet limit. Real behavior depends on TCP/TLS/HTTP-version setup, server configuration, and competing response bytes. Treat 14 KB as an order-of-magnitude budget; measure LCP and HTML transfer time on representative connections to choose the actual size.
- Load non-critical CSS with `media="print"` swap or `<link rel="preload">` + `onload` swap.
- Re-generate when the design changes; stale critical CSS is worse than none.

Delay-until-interaction for third-party scripts

For scripts that aren't needed for first paint — chat widgets, analytics, A/B testing, marketing tags — defer execution until the first user interaction (scroll, click, key). This trades some attribution accuracy for substantial INP and CPU savings on first load. Most performance plugins offer this as a configurable strategy; Perfmatters, FlyingScripts, and WP Rocket all implement it.

---

20. Page builders, themes, and DOM bloat

Page builders are not automatically slow, but they often raise the performance cost floor.

Review:

- DOM node count;
- wrapper depth;
- inline CSS volume;

- global CSS/JS frameworks;
- runtime layout engines;
- animation libraries;
- duplicate widgets/extensions;
- builder assets loaded on pages that do not use them.

Strategic rule:

> A page builder is a performance budget decision, not a moral choice.

Use builders where editorial flexibility justifies the cost. Prefer lean templates or native blocks for high-traffic, performance-critical pages where flexibility is less important.

---

21. Search, sitemaps, and large content sets

Search

WordPress database search does not scale well for large, complex search experiences.

Consider Elasticsearch/OpenSearch when:

- search traffic is high;
- filters/facets are complex;
- content volume is large;
- relevance ranking matters;
- SQL search creates load.

Sitemaps

At scale:

- paginate sitemaps;
- cache generated output;
- avoid generating deep archives on every request;
- exclude unnecessary post types/taxonomies;
- pre-generate where appropriate.

---

22. Current WordPress platform features and upgrade checks (verified against WordPress 7.0, 2026-06-14)

This section collects current WordPress Core and Performance Lab capabilities that affect a 2026 performance stack. Use it as an upgrade-check companion to the layer-specific guidance above, not as a separate appendix of optional add-ons.

Last verified against WordPress 7.0 on 2026-06-14.

Performance Lab feature plugins (verified against the plugin directory on 2026-06-14)

Performance Lab is the Core team's feature-plugin discovery and management layer. As features mature inside the Performance Lab ecosystem they either graduate into Core or remain as standalone plugins; the catalog rotates over time. Verify the current featured plugins on the Performance Lab plugin page before recommending any specific module.

Featured plugins as of 2026-06-14 include:

- Embed Optimizer — improves embeds that otherwise add third-party request and rendering cost.
- Enhanced Responsive Images — refines responsive image sizing decisions, including lazy-loaded images.
- Image Placeholders — formerly Dominant Color Images; uses a CSS background placeholder while an image loads.
- Image Prioritizer — automates `fetchpriority` and related loading decisions for likely LCP candidates.
- Instant Back/Forward — improves browser back/forward-cache behavior where applicable.
- Modern Image Formats — formerly named for WebP uploads; stores additional WebP/AVIF versions of uploaded images and serves modern formats to supporting browsers.
- Optimization Detective — opt-in real-user measurement that informs Core/Performance Lab decisions; dependency for some other featured plugins.
- Performant Translations, Speculative Loading, and View Transitions — experimental — additional featured plugins whose relevance depends on the site and rollout risk.

Recommendation: evaluate Performance Lab on staging, look at the current Featured Plugins list at the verification time, and adopt specific plugins by name only when they match the site's documented performance need. The catalog changes over time; for example, Performance Lab 4.1.0 removed Web Worker Offloading from the featured list.

Speculation Rules / Speculative Loading (Core in WordPress 6.8)

WordPress 6.8 (April 2025) merged speculative-loading support into Core via the Speculation Rules API. Core's effective default is `prefetch` with `conservative` eagerness, enabled on frontend requests only when pretty permalinks are on and the user is logged out. Core does not ship a dedicated settings UI for speculation rules in 6.8; customization happens through filters such as `wp_speculation_rules_configuration` and `wp_load_speculation_rules`, plus block-level CSS classes like `no-prefetch` and `no-prerender`.

The standalone Speculative Loading plugin extends Core with a plugin-provided admin screen and more aggressive modes, including `prerender`. When you opt into prerender, audit analytics SDKs for prerender-aware behavior and confirm that prerendered visits do not double-count or fire third-party scripts on hidden documents.

Enterprise considerations: edge cache hit ratio can rise on sites with predictable navigation patterns because prefetches may hit cache before the eventual navigation. Confirm session-keyed and personalized pages are excluded from speculative loading with Core filters, `no-prefetch` / `no-prerender` classes, or plugin-provided exclusion controls. Do not use older `data-prefetch="false"` wording as the recommended Core opt-out mechanism.

WP 6.4+ and 6.6+ Options API improvements

Already covered in §12: WordPress 6.4 introduced autoload helper APIs such as `wp_set_option_autoload()`, `wp_set_options_autoload()`, and `wp_set_option_autoload_valu`. WordPress 6.6 then expanded the stored autoload vocabulary (on, `off`, `auto`, `auto-on`, `auto-off`), added `wp_autoload_values_to_autoload()`, introduced large-option autoload heuristics, and added the Site Health autoload warning.

WordPress 6.9 performance deltas

WordPress 6.9 shipped on December 2, 2025. The most material 6.9 performance changes for this guide are:

- Frontend loading: the WordPress 6.9 Frontend Performance Field Guide covers script `fetchpriority`, footer script-module printing, emoji script-module changes, block stylesheet handling, hidden-block asset omission, template enhancement output buffering, RSS feed caching, video block CLS fixes, and related frontend work.
- WP-Cron spawn timing: Core now spawns WP-Cron at shutdown rather than earlier in the request lifecycle, reducing user-facing latency for request-triggered cron. Sites with a verified external cron runner remain the preferred high-traffic pattern.

- Query/cache changes: the WordPress 6.9 Field Guide should be checked before relying on plugin code that assumes older query-cache key behavior or intercepts Core caching internals.

Treat this as a delta inventory, not a substitute for measurement: verify impact on the specific site with the same baseline and rollback discipline used elsewhere in this guide.

WordPress 7.0 performance and compatibility deltas

WordPress 7.0 was released on May 20, 2026 and was the current active branch verified on 2026-06-14. The most relevant 7.0 changes for this performance guide are compatibility and operational-risk changes rather than a single universal speed feature:

- PHP support baseline: WordPress 7.0 drops support for PHP 7.2 and 7.3. The new minimum supported PHP version is 7.4.0, while PHP 8.3 remains the recommended baseline for modern performance work. For performance work, treat PHP 7.4 as a compatibility floor, not an optimization target; benchmark on the production-intended PHP 8.x runtime when possible.
- AI Client, Client-side Abilities, and Connectors: WordPress 6.9 introduced the server-side Abilities API as an AI-building-block foundation. WordPress 7.0 adds the WP AI Client, client-side Abilities package, and a Connectors screen/API. These features are infrastructure and extensibility surfaces, not automatic frontend performance improvements. If a site enables AI-backed workflows, MCP adapter/connectors, or connector plugins, include external API latency, timeout behavior, credential scoping, caching, and background processing in the performance review.
- Modernized admin/editor surfaces: Admin view transitions, Command Palette access, Font Library changes, Visual Revisions, and the iframed editor can improve operator experience but may change the JavaScript, CSS, and editor-plugin compatibility profile of admin screens. Re-test slow editorial workflows after upgrading.
- Real-time collaboration did not ship in Core 7.0: Do not attribute editor load, memory, or server-concurrency behavior to Core real-time collaboration in WordPress 7.0. The feature was removed before release because of concerns including race conditions, server load, memory efficiency, and recurring bugs.

Treat 7.0 guidance as an upgrade-review prompt: verify plugin/theme compatibility, PHP runtime behavior, admin/editor regressions, and any AI/connector network paths with measurements from the target environment.

Auto-image-sizes and `fetchpriority`

WP 6.3+ adds `fetchpriority="high"` to the LCP image when it can identify one (typically the first large in-content image). For most sites this removes the need for manual LCP preload — verify in DevTools before adding a preload yourself.

23. WooCommerce and dynamic commerce flows

WooCommerce sites often mix cacheable marketing/catalog pages with uncacheable cart, checkout, and account flows.

Checklist:

- Cache anonymous catalog and marketing pages aggressively.
- Bypass cart, checkout, and account pages.
- Audit cart fragments and AJAX calls.
- Avoid unnecessary session creation for anonymous visitors.
- Optimize product archive queries and filters.
- Monitor Action Scheduler (see §16).
- Avoid expensive personalization on every page view.
- Test logged-in, logged-out, cart, checkout, coupon, and account flows after cache changes.

---

24. Multisite

Multisite is not inherently slow. It amplifies shared-resource pressure.

Review:

- network-activated plugins;
- per-site options and transients;
- network-wide ("site") transients — `_site_transient_{name}` keys on the network options table — which behave like transients but with network scope and the same fallback semantics described in §13;
- object-cache memory and eviction behavior;
- database query volume;
- `switch_to_blog()` use (avoid in loops);
- domain mapping and redirects;
- site-specific vs network-wide bottlenecks.

Avoid heavy network-wide logic that every site inherits whether it needs it or not.

---

25. High-traffic events and load testing

High-traffic events require pre-work.

Preflight

- Identify expected traffic source and timing.
- Identify landing pages.
- Confirm edge cache hit ratio.
- Warm caches.
- Remove query-string cache fragmentation.
- Freeze risky deploys.
- Move cron/heavy jobs away from peak windows.
- Monitor origin, database, object cache, CDN, error rates.
- Prepare rollback and escalation.

Load testing

Only load test with approval.

```
k6 run --vus 100 --duration 5m script.js
```

Record:

- requests per second;
- p50/p95/p99 latency;
- error rate;
- cache hit ratio;
- origin CPU/memory;
- PHP workers;
- database load;
- object-cache hit rate;
- external dependency latency.

---

26. Measurement tools

Browser/lab tools

- Lighthouse;
- PageSpeed Insights;
- WebPageTest;
- Chrome DevTools;
- performance profiles;
- network waterfalls.

Field/user tools

- Chrome User Experience Report (CrUX);
- `web-vitals` JavaScript library for in-page RUM;
- Real User Monitoring providers (Scanfully, Sentry, New Relic Browser, etc.);
- analytics performance timing;
- Core Web Vitals reports.

WordPress/backend tools

Availability check. `wp profile` and `wp doctor` are not bundled with every WP-CLI install; they ship as separate command packages. Check availability before using them:

```
wp cli has-command profile && echo "profile available"
wp cli has-command doctor && echo "doctor available"
# Install if missing and your environment allows package installs:
# wp package install wp-cli/profile-command
# wp package install wp-cli/doctor-command
```

If neither command nor package install is available (common on managed hosts), fall back to Query Monitor, APM traces, host-provided profilers, or in-code `microtime()` timers.

- Query Monitor (useful across both general and enterprise guide contexts);
- WP-CLI profile (`wp profile stage`, `wp profile hook`) when the package is available;
- WP-CLI doctor (`wp doctor check`) when the package is available;
- PHP logs;
- custom timers;
- New Relic/APM;

- host/platform metrics;
- CDN logs/analytics;
- object-cache stats.

PHP timing example

```
function my_custom_function() {
    $start_time = microtime( true );

    // Code to be measured here.

    $execution_time = microtime( true ) - $start_time;
   error_log( 'my_custom_function took ' . $execution_time . ' seconds' );
}
```

Query Monitor custom timer

```
do_action( 'qm/start', 'mytimer' );

foreach ( range( 1, 10 ) as $i ) {
    function_to_measure_time_spent( $i );
}

do_action( 'qm/stop', 'mytimer' );
```

New Relic example

```
// Guard so the code is safe whether the New Relic PHP agent is loaded or not.
if ( extension_loaded( 'newrelic' ) && function_exists( 'newrelic_name_transaction'
    // Name the current transaction so it appears as a distinct entry
    // in the New Relic APM transaction list. Avoid per-user or per-
URL names.
    newrelic_name_transaction( 'myplugin/expensive-path' );
}

// Optional: add custom attributes for filtering and querying in NRQL.
if ( function_exists( 'newrelic_add_custom_parameter' ) ) {
   newrelic_add_custom_parameter( 'plugin_context', 'myplugin_expensive_path' );
}
```

Use `newrelic_name_transaction()` to give the current request a stable, low-cardinality name in APM. The New Relic transaction-start API takes a New Relic application name, not a transaction name, and is reserved for advanced transaction-lifecycle patterns such as queue workers manually ending one transaction and starting another.

27. Decision trees

These are TRACE in checklist form (see §3): follow the signal, narrow the pattern, change one thing, verify.

High TTFB on anonymous public page

1. Should the page be full-page cached?
2. Is it a cache hit?
3. If miss/bypass, inspect cookies, headers, query strings, TTL, purge behavior.
4. If truly uncacheable, profile PHP, database, object cache, external calls.
5. Verify with repeated warm/cold measurements.

High TTFB on logged-in/admin page

1. Full-page cache likely does not apply.
2. Profile hooks and callbacks.
3. Check query count and slow queries.
4. Check autoloaded options (§12 — use the 6.6-aware query).
5. Check object-cache hit rate.
6. Check external HTTP calls.
7. Check cron/background jobs.

Poor LCP

1. Is TTFB high?
2. What is the LCP element?
3. Is the LCP image optimized and prioritized (`fetchpriority`)?
4. Is CSS/JS blocking render?
5. Are fonts delaying visible text (FOIT)?
6. Are third parties delaying the first view?

Poor INP

1. Capture main-thread profile.
2. Identify long tasks (over 50 ms).
3. Attribute scripts to plugins/themes/third parties.
4. Remove, defer, delay-until-interaction (§19), or split work.
5. Reduce DOM/layout complexity.

Poor cache hit ratio

1. Are URLs fragmented by query strings?
2. Are cookies forcing variation or bypass?
3. Are TTLs too short?
4. Are purges too broad/frequent?
5. Are personalized pages mixed with public pages?
6. Are REST/AJAX calls bypassing the intended cache strategy?

Slow database

1. Identify the slow query.
2. Run `EXPLAIN`.
3. Check indexes and rows scanned.
4. Check for unbounded query patterns.
5. Check meta/taxonomy misuse.
6. Cache result if safe.
7. Consider search/offloaded storage if SQL is the wrong tool.

Site fails under traffic but is fine when quiet

1. Which shared resource saturates first — PHP workers, database connections, object-cache memory, CDN egress?
2. Are cache misses concentrated on a small set of URLs?
3. Are background jobs running during the peak?
4. Is there a stampede pattern on a hot key?

---

28. Developer anti-pattern checklist

Avoid or review carefully:

- global plugin initialization doing expensive work;
- remote HTTP calls during render;
- `posts_per_page => -1`;
- broad meta queries;
- large `NOT IN` lists;
- leading-wildcard `LIKE`;
- high-cardinality transients without persistent object cache;
- large autoloaded options (and pre-6.6 review queries that miss the new vocabulary);
- object-cache keys with poor reuse;
- cache regeneration without locks;
- `admin-ajax.php` polling;
- frontend scripts enqueued globally;
- multiple plugins modifying the same CSS/JS/cache layer;
- page-builder templates on performance-critical landing pages without budget;
- full-site cache purges for small changes;
- treating Lighthouse score as the only goal;
- optimizing before measuring.

Common transient misconceptions

These are myths worth flagging — they appear in the sibling transients reference and routinely catch developers:

- "Transients are always memory cache." No — without persistent object caching, they're database rows.
- "`WP_CACHE` enables Redis or Memcached." No — `WP_CACHE` relates to `advanced-cache.php`. Persistent object caching is `object-cache.php`.
- "A transient will always exist until its expiration time." No — expiration is a maximum lifetime, not a minimum. Cache eviction, flushes, and storage pressure can remove a transient early. Always have a regeneration path.
- "Expired transients always disappear immediately." No — on database-backed sites, expired transient rows can sit in `wp_options` until accessed or until a cleanup runs.
- "Transients are safe for unlimited per-user/per-request data." No — high-cardinality keys create either huge database growth or low cache hit rates, depending on backend.
- "Deleting expired transients is performance optimization." Sometimes it helps hygiene, but it does not fix code that keeps creating excessive transient rows.

---

29. Definition of done

A performance fix is done when:

- the bottleneck was measured;
- the fix targets that bottleneck directly;
- before/after measurements use the same URL, user state, cache state, and environment;
- cache behavior is safe and intentional;
- no private content is cached publicly;
- critical user flows still work;
- error logs are clean;
- monitoring can detect regression;
- remaining risks are documented.

Client/stakeholder reporting

A performance pass is not communicated by score deltas alone. Frame the work as:

- What was slow?
- What work was reduced, cached, or moved?
- What changed in measured results (at the same percentile, same user state)?
- What risks remain?
- What should be monitored next?

Area	Before	After	Impact	Next action
Homepage TTFB	900 ms	180 ms	HTML cache now hitting	Watch cache hit ratio
Product LCP	4.2 s	2.5 s	Hero image re-sized/preload	Audit remaining images
Admin order screen	6.8 s	3.1 s	Slow query removed	Monitor during peak

30. Source map and external references

Local source and companion files

- General checklist: `wordpress-performance-optimization-checklist.md`
- Enterprise checklist: `enterprise-performance-operational-checklist.md`
- Transients reference: `REFERENCE-WP-Transients-Persistent-Object-Cache.md`
- Source-correction audit trail: `docs/source-corrections.md`

WordPress.org developer documentation

- Transients API
- `get_transient()`
- `set_transient()`
- `WP_Object_Cache`
- `wp_start_object_cache()`
- `wp_set_option_autoload()`
- `wp_set_options_autoload()`
- `wp option set-autoload` (WP-CLI)
- WP_Query class reference
- Advanced Administration Handbook — Cache
- Advanced Administration Handbook — Performance
- Plugin Handbook — Hooking WP-Cron into the system task scheduler

WordPress Core blog posts

- Options API: Disabling autoload for large options (June 2024)
- New option functions in 6.4 (October 2023)
- WordPress 6.6 Performance Improvements
- Speculative Loading in 6.8 (March 2025)
- WordPress 6.8 Performance Improvements (April 2025)
- WordPress 6.9 Frontend Performance Field Guide
- WordPress 6.9 Field Guide
- WordPress 7.0 Field Guide
- Dropping support for PHP 7.2 and 7.3
- Real-time collaboration will not ship in WordPress 7.0
- WordPress release archive

Performance Lab and standalone plugins

- Performance Lab
- Speculative Loading
- Image Placeholders / Dominant Color Images

Web platform / Core Web Vitals

- web.dev — Learn Core Web Vitals
- web.dev — Web Vitals overview
- web.dev — Interaction to Next Paint
- web.dev — Defining Core Web Vitals thresholds
- `web-vitals` JavaScript library
- MDN — Critical rendering path
- MDN — `font-display`

Advanced Admin Handbook PR

- WordPress/Advanced-administration-handbook PR #485